

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验 |LAB1

基于 MPI 的并行矩阵乘法

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 吴迪、刘学正

起 始 日 期: 2026 年 4 月 7 日

结 束 日 期: 2026 年 4 月 7 日

实 验 地 点: B202

目录

1 实验目的 **2**

2 实验内容 **2**

3 实验环境 **2**

4 算法设计与实现 **2**
4.1 测试用例设计与控制变量声明 . . . 2
4.1.1 契合基准测试标准 . . . 2
4.1.2 纯粹的复杂度博弈分析 . . . 2
4.1.3 消除边界切割噪声 . . . 3
4.2 设计思路 . . . 3
4.2.1 数据分布 . . . 3
4.2.2 通信流程 . . . 3
4.3 核心代码实现 . . . 3
4.3.1 矩阵分配与初始化 . . . 3
4.3.2 矩阵分发（点对点通信） . . . 4
4.3.3 本地矩阵乘法计算 . . . 4
4.3.4 结果收集与计时 . . . 5

5 性能测试与分析 **5**
5.1 测试结果 . . . 5
5.2 加速比分析 . . . 6
5.3 性能深度分析 . . . 7
5.3.1 小规模矩阵的通信过载现象 . . . 7
5.3.2 大规模矩阵的 P=2 性能异常与后续扩展性 . . . 8

6 讨论题 **8**
6.1 问题一：在内存受限情况下，如何进行大规模矩阵乘法计算? . . . 8
6.1.1 核外计算 (Out-of-Core Computation) 策略 . . . 9
6.1.2 矩阵分块算法 . . . 9
6.1.3 分布式核外计算 . . . 10
6.2 问题二：如何提高大规模稀疏矩阵乘法性能? . . . 10
6.2.1 稀疏存储格式 . . . 10
6.2.2 负载不均衡问题 . . . 10
6.2.3 并行环境下的负载均衡策略 . . . 11
6.2.4 其他优化技术 . . . 12

7 实验总结 **12**

1 实验目的

- 掌握 MPI 程序的编译和运行方法。
- 理解 MPI 点对点通信的基本原理。
- 了解 MPI 程序的 GDB 调试流程。

2 实验内容

- 使用 MPI 点对点通信实现并行矩阵乘法。
- 设置进程数量（1∼16）及矩阵规模（128∼2048）。
- 根据运行时间，分析程序的并行性能。

3 实验环境

- **操作系统**: WSL2 Ubuntu 22.04
- **编译器**: GCC (GNU Compiler Collection)
- **MPI 实现**: OpenMPI (libopenmpi-dev, openmpi-bin)

4 算法设计与实现

4.1 测试用例设计与控制变量声明

本实验在性能评测中统一采用 $N \times N$ 的方阵作为测试输入，即将矩阵乘法的三个维度参数简化为 $m = n = k = N$。这一设计决策基于以下三个关键考量：

4.1.1 契合基准测试标准

实验指导书提供的性能测试表格中，”矩阵规模”仅列出了单一维度（128、256、512、1024、2048）。为严格遵循指导书的测试规范并确保数据可直接填入标准表格，本实验将输入参数统一设置为方阵形式。这种简化不仅使测试结果具有横向可比性，也便于与课程提供的参考数据进行对照验证。

4.1.2 纯粹的复杂度博弈分析

MPI 点对点通信并行矩阵乘法的核心性能瓶颈在于 $O(N^3)$ 的计算收益与 $O(N^2)$ 的通信开销之间的博弈。若采用比例悬殊的非方阵（如 $m \gg n$ 或 $n \gg k$），计算量与通信量的比例会发生剧烈扭曲，引入”形状变量”这一额外干扰因素。例如，对于 1000×100 的矩阵 A 与 100×1000 的矩阵 B，计算量为 10^8，而通信量取决于 A 的行数和 B 的完整传输，比例关系与方阵情形截然不同。这种形状差异会严重干扰对”加速比随规模倍增规律”的直观观察与分析。采用方阵可确保计算复杂度与通信复杂度保持稳定的 $N^3 : N^2$ 比例关系，从而测得最纯粹的并行效率。

4.1.3 消除边界切割噪声

在使用基础的 MPI_Send 和 MPI_Recv 进行点对点通信时，行块划分策略要求矩阵行数 m 能被进程数 P 完美整除。若 m 不能整除 P，则必须引入繁琐的边界不对齐处理逻辑：部分进程需处理 $\lceil m/P \rceil$ 行，而其他进程仅处理 $\lfloor m/P \rfloor$ 行，这不仅会导致微小的负载不均衡，还会产生额外的条件判断和数据余数处理代码执行开销（噪声）。本实验统一采用 2 的幂次方方阵（$N = 2^i$），结合进程数同样为 2 的幂次（$P = 2^j$），可完美实现内存对齐和均匀切分，从而消除边界处理噪声，测得最真实的通信损耗与并行加速效果。

4.2 设计思路

本实验采用**行块划分策略 (Row-wise Block Distribution)** 实现并行矩阵乘法。核心思想是将矩阵 A 按行均匀划分给各个进程，每个进程负责计算结果矩阵 C 的对应行块。

4.2.1 数据分布

设矩阵规模为 $N \times N$，进程数为 P，则：

- 矩阵 A: 按行划分，每个进程获得 N/P 行（本实验中 N 和 P 均为 2 的幂次，可整除）
- 矩阵 B: 完整广播给所有进程（每个进程都需要完整的 B 进行计算）
- 矩阵 C: 各进程计算对应的行块，最后收集到 Root 进程（0 号进程）

4.2.2 通信流程

本实验严格使用点对点通信函数 MPI_Send 和 MPI_Recv，通信流程如下：

1. **分发矩阵 B**: Root 进程使用 MPI_Send 将完整矩阵 B 发送给所有 Worker 进程（进程 1 至 $P-1$），Worker 使用 MPI_Recv 接收。
2. **分发矩阵 A 的行块**: Root 进程将 A 的第 i 个行块（第 $i \times (N/P)$ 至 $(i+1) \times (N/P) - 1$ 行）使用 MPI_Send 发送给进程 i。
3. **本地计算**: 各进程独立执行本地矩阵乘法 local_C = local_A $\times$ B。
4. **收集结果**: Worker 进程使用 MPI_Send 将计算结果 local_C 发回 Root，Root 使用 MPI_Recv 依次接收并组装成完整结果矩阵 C。

4.3 核心代码实现

4.3.1 矩阵分配与初始化

```c
// 矩阵分配函数
double* allocate_matrix(int rows, int cols) {
    double *matrix = (double*)malloc(rows * cols * sizeof(double));
    if (matrix == NULL) {
        fprintf(stderr, "Error: Failed to allocate matrix memory\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    return matrix;
}
```

```

// 矩阵初始化函数（使用固定种子保证可重复性）
void init_matrix(double *matrix, int rows, int cols, int seed) {
    srand(seed);
    for (int i = 0; i < rows * cols; i++) {
        matrix[i] = (double)(rand() % 100) / 10.0;
    }
}
```

4.3.2 矩阵分发（点对点通信）

```
// 分发矩阵 B（所有进程都需要）
B = allocate_matrix(N, N);

if (rank == 0) {
    // Root 发送 B 给所有其他进程
    for (int p = 1; p < size; p++) {
        MPI_Send(B, N * N, MPI_DOUBLE, p, 0, MPI_COMM_WORLD);
    }
} else {
    // 其他进程接收 B
    MPI_Recv(B, N * N, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}

// 分发矩阵 A 的行块
local_A = allocate_matrix(rows_per_proc, N);

if (rank == 0) {
    // Root 保留自己的行块
    memcpy(local_A, A, rows_per_proc * N * sizeof(double));

    // 发送其他行块给其他进程
    for (int p = 1; p < size; p++) {
        int start_row = p * rows_per_proc;
        MPI_Send(&A[start_row * N], rows_per_proc * N, MPI_DOUBLE,
                 p, 1, MPI_COMM_WORLD);
    }
} else {
    // 其他进程接收 A 的行块
    MPI_Recv(local_A, rows_per_proc * N, MPI_DOUBLE, 0, 1,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}
```

4.3.3 本地矩阵乘法计算

```
void matrix_multiply(double *A, double *B, double *C,
                     int rows_A, int cols_A, int cols_B) {
    for (int i = 0; i < rows_A; i++) {
        for (int j = 0; j < cols_B; j++) {
            double sum = 0.0;
            for (int k = 0; k < cols_A; k++) {
                sum += A[i * cols_A + k] * B[k * cols_B + j];
            }
            C[i * cols_B + j] = sum;
        }
    }
```

```
}
```

4.3.4 结果收集与计时

```
// 本地计算
matrix_multiply(local_A, B, local_C, rows_per_proc, N, N);

// 收集结果到 Root
if (rank == 0) {
    // Root 复制自己的结果
    memcpy(C, local_C, rows_per_proc * N * sizeof(double));

    // 接收其他进程的结果
    for (int p = 1; p < size; p++) {
        int start_row = p * rows_per_proc;
        MPI_Recv(&C[start_row * N], rows_per_proc * N, MPI_DOUBLE,
                 p, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
    end_time = MPI_Wtime();
    printf("Size: %d, Procs: %d, Time: %.4f s\n", N, size, end_time - start_time);
} else {
    // 其他进程发送结果给 Root
    MPI_Send(local_C, rows_per_proc * N, MPI_DOUBLE, 0, 2, MPI_COMM_WORLD);
}
```

5 性能测试与分析

5.1 测试结果

根据运行结果，不同进程数和矩阵规模下的运行时间如下表所示：

表 1: 矩阵乘法运行时间 (单位: 秒)

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.0063	0.0762	0.9434	7.1551	58.7634
2	0.0034	0.0410	0.8475	7.8624	66.3320
4	0.0029	0.0258	0.4372	3.9162	34.7804
8	0.0036	0.0197	0.2366	1.7962	25.7218
16	0.0290	0.0392	0.2835	1.2938	14.8895

```
All tests completed!
Results saved to: results.txt

Summary:
# Format: Size: <N>, Procs: <P>, Time: <T> s
Size: 128, Procs: 1, Time: 0.0063 s
Size: 256, Procs: 1, Time: 0.0762 s
Size: 512, Procs: 1, Time: 0.9434 s
Size: 1024, Procs: 1, Time: 7.1551 s
Size: 2048, Procs: 1, Time: 58.7634 s
Size: 128, Procs: 2, Time: 0.0034 s
Size: 256, Procs: 2, Time: 0.0410 s
Size: 512, Procs: 2, Time: 0.8475 s
Size: 1024, Procs: 2, Time: 7.8624 s
Size: 2048, Procs: 2, Time: 66.3320 s
Size: 128, Procs: 4, Time: 0.0029 s
Size: 256, Procs: 4, Time: 0.0258 s
Size: 512, Procs: 4, Time: 0.4372 s
Size: 1024, Procs: 4, Time: 3.9162 s
Size: 2048, Procs: 4, Time: 34.7804 s
Size: 128, Procs: 8, Time: 0.0036 s
Size: 256, Procs: 8, Time: 0.0197 s
Size: 512, Procs: 8, Time: 0.2366 s
Size: 1024, Procs: 8, Time: 1.7962 s
Size: 2048, Procs: 8, Time: 25.7218 s
```

图 1: 1 至 8 进程运行结果截图

```
root@fortune:~/ParallelComputingLab/lab1/code# mpirun --allow-run-as-root --oversubscribe -n 16 ./mpi_v1 128
Matrix multiplication: N=128, Processes=16
Size: 128, Procs: 16, Time: 0.0290 s
root@fortune:~/ParallelComputingLab/lab1/code# mpirun --allow-run-as-root --oversubscribe -n 16 ./mpi_v1 256
Matrix multiplication: N=256, Processes=16
Size: 256, Procs: 16, Time: 0.0392 s
root@fortune:~/ParallelComputingLab/lab1/code# mpirun --allow-run-as-root --oversubscribe -n 16 ./mpi_v1 512
Matrix multiplication: N=512, Processes=16
Size: 512, Procs: 16, Time: 0.2835 s
root@fortune:~/ParallelComputingLab/lab1/code# mpirun --allow-run-as-root --oversubscribe -n 16 ./mpi_v1 2048
Matrix multiplication: N=2048, Processes=16
Size: 2048, Procs: 16, Time: 14.8895 s
root@fortune:~/ParallelComputingLab/lab1/code# mpirun --allow-run-as-root --oversubscribe -n 16 ./mpi_v1 1024
Matrix multiplication: N=1024, Processes=16
Size: 1024, Procs: 16, Time: 1.2938 s
```

图 2: 16 进程运行结果截图

5.2 加速比分析

以单进程为基准，计算各配置下的加速比 $S_p = T_1/T_p$:

表 2: 并行加速比

进程数	矩阵规模				
	128	256	512	1024	2048
1	1.00	1.00	1.00	1.00	1.00
2	1.85	1.86	1.11	0.91	0.89
4	2.17	2.95	2.16	1.83	1.69
8	1.75	3.87	3.99	3.98	2.28
16	0.22	1.94	3.33	5.54	3.95

5.3 性能深度分析

5.3.1 小规模矩阵的通信过载现象

观察表格数据可以发现一个显著现象：对于小规模矩阵（128、256），当进程数增加到 16 时，运行时间反而远大于单进程。具体数据如下：

- 矩阵规模 128：单进程 0.0063s，16 进程 0.0290s（加速比仅 0.22，即" 负加速"，耗时增加约 4.6 倍）
- 矩阵规模 256：单进程 0.0762s，16 进程 0.0392s（加速比仅 1.94，远低于理想的 16 倍）

原因分析：
这一现象深刻体现了 **阿姆达尔定律 (Amdahl's Law)** 的限制。根据阿姆达尔定律：

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}}$$

其中 f 为可并行化部分的比例，p 为进程数。当串行部分比例 $(1-f)$ 过高时，增加进程数无法有效提升性能。

对于小规模矩阵，MPI 点对点通信的各类开销占据了主导地位：

1. **序列化与打包开销**: `MPI_Send` 在发送前需要将数据序列化打包，对于小数据量，这个固定开销在总时间中占比显著。
2. **网络/内存拷贝开销**: MPI 通信涉及多次内存拷贝（用户缓冲区到系统缓冲区，再到接收方缓冲区），这些拷贝操作的时间与数据量成正比，但对于小矩阵，固定开销（启动延迟 latency）远大于数据传输时间。
3. **点对点通信的串行瓶颈**: 本实现中 Root 进程必须依次执行 15 次 `MPI_Send`（分发 B 和 A 的行块）和 15 次 `MPI_Recv`（收集结果），共 30 次通信操作。每次通信的启动延迟约为微秒级，累积后可达毫秒级，远超小矩阵的计算时间（$128^3 \approx 2.1 \times 10^6$ 次浮点运算，现代 CPU 可在微秒级完成）。
4. **进程创建与同步开销**: 16 个 MPI 进程的创建、初始化和最终同步（`MPI_Finalize`）也需要一定时间，在小规模测试中不可忽略。

综上所述，对于小规模矩阵，通信开销（串行部分）远大于计算开销（并行部分），导致串行部分比例 $(1-f)$ 极高，加速比严重退化甚至出现负加速。

5.3.2 大规模矩阵的 P=2 性能异常与后续扩展性

对于大规模矩阵（1024、2048），观察到一个值得注意的现象：

- 矩阵规模 1024：P=1 耗时 7.15s，P=2 耗时 7.86s（加速比 0.91，负加速）
- 矩阵规模 2048：P=1 耗时 58.7s，P=2 耗时 66.3s（加速比 0.89，负加速）

然而，随着进程数继续增加至 4、8、16，性能显著提升：

- 矩阵规模 1024：P=4 耗时 3.92s，P=8 耗时 1.80s，P=16 耗时 1.29s（加速比达 5.54）
- 矩阵规模 2048：P=4 耗时 34.8s，P=8 耗时 25.7s，P=16 耗时 14.9s（加速比达 3.95）

P=2 性能异常原因分析：

1. **Root 进程的内存带宽竞争**: 在 P=2 配置下，Root 进程（进程 0）承担了所有通信任务：发送完整矩阵 B（1024 规模下约 8MB，2048 规模下约 32MB）和 A 的半数行块给 Worker，同时接收 Worker 计算的半数结果。这些通信操作涉及大量内存拷贝，Root 进程在执行 `MPI_Send` 和 `MPI_Recv` 时需要频繁访问内存，与本地计算任务产生严重的内存带宽竞争。
2. **点对点通信的阻塞特性**: `MPI_Send` 和 `MPI_Recv` 是阻塞式通信函数。Root 进程在发送数据时必须等待接收方准备好接收缓冲区，在接收数据时必须等待发送方完成发送。这种阻塞特性导致 Root 进程在通信期间无法进行其他计算，形成同步等待瓶颈。
3. **单 Worker 的计算优势不足**: P=2 时，Worker 进程只分担了 50% 的计算任务，但其引入的通信开销（接收 B、接收 A 行块、发送结果）可能超过其计算收益。特别是 Root 进程仍需承担 50% 计算 + 100% 通信调度，负载反而更重。

P=4, 8, 16 扩展性改善原因：

1. **计算任务分摊效应**: 随着进程数增加，每个进程分摊的计算任务变小。P=16 时，每个进程只需计算 $N/16$ 行，计算量大幅减少。多核 CPU 的算力优势终于能够覆盖通信成本。
2. **通信与计算的相对比例改善**: 对于大规模矩阵，计算复杂度 $O(N^3)$ 远大于通信复杂度 $O(N^2)$。当 N 足够大时，计算时间主导总时间，增加进程数能有效减少计算时间，通信开销的相对比例降低。
3. **并行计算效率提升**: 多进程并行计算能够充分利用多核 CPU 的并行算力，每个进程独立执行本地矩阵乘法，互不干扰（除内存带宽竞争外）。

然而，P=16 的加速比（5.54 和 3.95）仍未达到理想的线性加速（16 倍），这仍然受限于：

- Root 进程的串行通信调度（阿姆达尔定律的串行部分）
- 多进程的内存带宽竞争
- MPI 进程间的同步开销

6 讨论题

6.1 问题一：在内存受限情况下，如何进行大规模矩阵乘法计算？

当矩阵规模超过可用内存容量时，无法将整个矩阵一次性加载到内存中。此时需要采用 **核外计算 (Out-of-Core Computation)** 策略，结合矩阵分块算法来实现大规模矩阵乘法。

6.1.1 核外计算 (Out-of-Core Computation) 策略

核外计算的核心思想是将矩阵存储在外存（磁盘/SSD）上，按需将数据块调入内存进行计算：

1. **矩阵分块存储**: 将大矩阵划分为适合内存容量的块（Block），每个块存储为独立的磁盘文件。例如，将 $N \times N$ 矩阵划分为 $B \times B$ 的块矩阵，共有 $(N/B)^2$ 个块。块大小 B 的选择应确保三个块（A_{block}、B_{block}、C_{block}）能同时放入可用内存。

2. **分块调度策略**: 采用”按需加载”策略，每次只加载参与当前计算的两个块到内存。计算完成后，将结果块写入磁盘，释放内存空间加载下一组块。调度顺序应优化以最大化数据重用率。

3. **计算与 I/O 的重叠**: 这是核外计算的关键优化技术：

   - **双缓冲 (Double Buffering)**: 维护两个缓冲区，在计算当前块时，异步预取下一块数据到另一个缓冲区。计算完成后直接切换缓冲区，无需等待 I/O。
   - **异步 I/O**: 使用异步文件读写接口（如 POSIX aio、Windows OVERLAPPED I/O），在后台执行 I/O 操作，主线程继续计算。
   - **流水线设计**: 将计算过程设计为流水线，I/O 阶段、计算阶段、写回阶段并行执行，最大化资源利用率。

4. **内存映射文件 (mmap)**: 使用操作系统提供的内存映射机制，将磁盘文件映射到虚拟内存地址空间。操作系统自动处理页面调入/调出，简化编程模型并利用页面缓存优化。

6.1.2 矩阵分块算法

分块算法不仅适用于核外计算，也能优化分布式内存系统的通信效率：

1. **SUMMA 算法 (Scalable Universal Matrix Multiplication Algorithm)**:

   - SUMMA 是最通用的分块矩阵乘法算法，支持任意处理器网格配置（$P \times Q$）。
   - **核心思想**: 将矩阵 A 和 B 划分为块矩阵，每次迭代广播 A 的一列块和 B 的一行块，各处理器计算本地块的乘积并累加。
   - **核外适配**: 每次迭代只需加载广播涉及的块，其他块保留在磁盘。通过合理设计块访问顺序，可最小化磁盘 I/O 次数。
   - **通信效率**: 广播操作利用优化的通信树，通信复杂度为 $O(N^2 \log p)$，低于点对点串行通信。

2. **Cannon 算法**:

   - 适用于 $p \times p$ 处理器网格，将矩阵划分为 $p \times p$ 个块。
   - 初始对齐阶段：A 的行块向左循环移位，B 的列块向上循环移位。
   - 迭代计算阶段：每步计算本地块乘积，然后 A 向左移位、B 向上移位。
   - 通信复杂度 $O(N^2/\sqrt{p})$，适合大规模并行系统。

3. **块大小选择原则**:

   - 块大小 B 应根据可用内存确定：$3 \times B^2 \times 8$ bytes $\leq$ 可用内存
   - 考虑缓存层次：B 应接近 L1/L2 缓存大小以最大化缓存命中率
   - 考虑 I/O 效率：较大的块可减少 I/O 次数，但增加单次 I/O 时间

6.1.3 分布式核外计算

在 MPI 并行环境下，核外计算需要协调多进程的 I/O 操作：

- 使用并行文件系统（如 Lustre、GPFS）提高 I/O 带宽
- 设计全局 I/O 调度策略，避免多进程同时访问同一磁盘区域
- 利用 MPI-IO 接口进行并行文件读写，支持集体 I/O 操作

6.2 问题二：如何提高大规模稀疏矩阵乘法性能？

稀疏矩阵中大部分元素为零，直接使用稠密矩阵算法会造成大量无效计算和存储浪费。提高稀疏矩阵乘法性能需要从存储格式、负载均衡和并行策略三个方面进行优化。

6.2.1 稀疏存储格式

选择合适的稀疏存储格式是提高性能的基础：

1. **CSR (Compressed Sparse Row)**:
   - 使用三个数组存储：`values`（非零元素值）、`col_indices`（列索引）、`row_ptr`（行指针）
   - 行指针数组记录每行非零元素的起始位置，支持高效的行访问
   - 存储开销：$O(\text{nnz} + N + 1)$，其中 nnz 为非零元素数量，N 为行数
   - **优势**: 大幅减少内存占用（只存储非零元素），减少无效计算（只遍历非零元素），提高访存带宽利用率
   - 适合行访问为主的操作，如 SpMV（稀疏矩阵-向量乘法）
2. **CSC (Compressed Sparse Column)**:
   - 与 CSR 类似，但按列压缩存储
   - 适合列访问为主的操作
3. **COO (Coordinate Format)**:
   - 存储每个非零元素的行索引、列索引和值
   - 存储开销：$O(3 \times \text{nnz})$
   - 适合矩阵构建阶段，易于添加/删除元素

6.2.2 负载不均衡问题

稀疏矩阵并行计算面临的核心挑战是 **负载不均衡 (Load Imbalance)**：

1. **问题根源**:
   - 稀疏矩阵的非零元素分布往往不均匀。例如，某些行可能包含大量非零元素（密集行），而其他行只有少量非零元素（稀疏行）。
   - 传统的” 按行均分” 策略假设每行计算量相同，但实际上每行的计算量取决于其非零元素数量。

- 例如：某行有 1000 个非零元素，另一行只有 10 个。若分配到不同进程，前者进程的计算量是后者的 100 倍。

2. **影响**:

- 负载最重的进程成为瓶颈，其他进程必须等待其完成
- 并行效率大幅下降，加速比远低于理想值
- 通信阶段也可能因数据量差异导致不均衡

6.2.3 并行环境下的负载均衡策略

1. **基于非零元素数量的一维划分**:

- 预先统计每行的非零元素数量 nnz_i
- 根据非零元素总数动态分配行数，使各进程处理的非零元素总数相近
- 目标函数：$\min \max_i \sum_{j \in \text{partition}_i} \text{nnz}_j$
- 实现方法：使用贪心算法或二分搜索确定最优划分点

2. **二维动态任务划分**:

- 将矩阵划分为二维块网格，每个块作为独立任务
- 根据块内非零元素数量分配任务权重
- 使用动态调度策略（Master-Worker 或工作窃取）分配任务
- 优势：更细粒度的负载均衡，适应不规则稀疏结构

3. **动态任务分配 (Master-Worker 模式)**:

- Master 进程维护任务队列（每个任务为一组行的计算）
- Worker 进程完成当前任务后，向 Master 请求新任务
- 任务粒度可根据非零元素密度动态调整
- 优点：自动适应负载变化，无需预先分析矩阵结构
- 缺点：增加通信开销（任务请求和分配）

4. **工作窃取 (Work Stealing)**:

- 每个进程维护本地任务队列
- 空闲进程从忙碌进程的队列中” 窃取” 部分任务
- 减少中心化调度开销，提高可扩展性
- MPI 实现需要额外的进程间通信协调

5. **图划分方法**:

- 将稀疏矩阵视为图：行/列为顶点，非零元素为边
- 使用图划分工具（如 METIS、ParMETIS）将顶点分配到各处理器
- 目标：最小化边切割（减少通信）同时平衡顶点权重（负载均衡）
- 特别适合不规则稀疏矩阵

6.2.4 其他优化技术

1. **通信优化**:
   - 压缩通信数据：只传输非零元素及其索引
   - 合并小消息：减少通信次数，降低启动延迟影响
   - 非阻塞通信：实现计算与通信重叠
2. **数据局部性优化**:
   - 尽量将相关数据分配到同一进程，减少跨进程数据依赖
   - 利用矩阵的局部性特征（如对角带状结构）
3. **使用成熟库**:
   - PETSc: 提供高效的稀疏矩阵并行计算功能
   - Trilinos: 支持多种稀疏格式和并行算法
   - Intel MKL Sparse: 针对 Intel 架构优化的稀疏计算库

7 实验总结

本实验通过实现基于 MPI 点对点通信的并行矩阵乘法，理解了以下内容：

1. MPI 点对点通信函数 `MPI_Send` 和 `MPI_Recv` 的使用方法及其阻塞特性
2. 行块划分策略的数据分布方式和通信流程设计
3. 并行程序性能测试方法与加速比分析
4. **小规模矩阵的通信过载现象**: 通信开销超过计算收益，导致加速比退化甚至负加速
5. **大规模矩阵的扩展性**: 多进程能有效加速，但受限于阿姆达尔定律和内存带宽竞争
6. 核外计算策略和稀疏矩阵负载均衡问题的深入讨论与理解